

Day 6 - Facilitation Guide

(Classes and Objects)

INDEX

- I. Recap of the previous session**
- II. Defining Classes**
- III. Creating Objects**
- IV. Class Attributes and Methods**
- V. Instance Attributes and Methods**

(2 hrs) ILT

I. Recap of the previous session

In the last session, we taught you many important concepts of Decorators in Python. Here's a brief introductory recap session covering the key concepts you've taught in the last session:

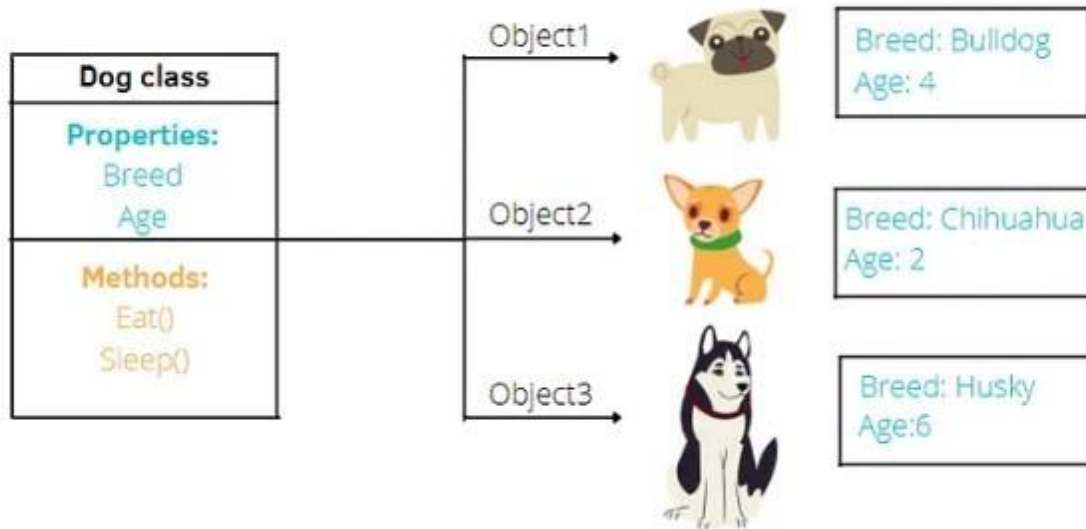
Understanding Decorators: Decorators in Python are a powerful tool that allows you to modify or extend the behavior of functions or methods without changing their actual code. They are commonly used to add functionality like logging, access control, memoization, and more.

Creating Decorators: Creating decorators in Python involves defining a function that takes another function as an argument, wraps it with some additional functionality, and returns the modified function. Decorators allow you to add behavior to functions or methods in a clean and reusable way.

Using Multiple Decorators: Using multiple decorators on a single function in Python allows you to stack functionality, where each decorator adds or modifies behavior. When multiple decorators are applied, they are executed in the order from the closest to the function definition outward. This means the decorator closest to the function name is applied first, and then the next outermost decorator, and so on.

By the end of this session, students will be able to define classes in Python, understanding the structure and syntax required to create reusable blueprints for objects. They will learn how to instantiate objects from these classes and explore the difference between class attributes, which are shared across all instances, and instance attributes, which are unique to each object. Additionally, students will understand and implement class methods and instance methods, recognizing how they function differently and their appropriate use cases within object-oriented programming.

II. Defining Classes



Defining classes is a fundamental concept in object-oriented programming (OOP). A class is a blueprint for creating objects (instances), and it encapsulates data for the object and methods to manipulate that data.

Components of a Class

- **Class Name:** The name you give to the class.
- **Attributes:** Variables that store data for the object.
- **Methods:** Functions that define the behaviors or actions the object can perform.
- **Constructor:** A special method used to initialize objects.

Syntax for Defining a Class in Python:

```
class ClassName:
    # Class Attribute (optional)
    class_attribute = "This is a class attribute"

    # Constructor (initializes the object)
    def __init__(self, attribute1, attribute2):
        # Instance Attributes
        self.attribute1 = attribute1
        self.attribute2 = attribute2

    # Method
    def method_name(self):
        # Perform an action
        print(f"Attribute1: {self.attribute1}, Attribute2: {self.attribute2}")
```

Explanation

- **ClassName:** The name of the class, which should start with an uppercase letter by convention.
- **class_attribute:** A variable shared among all instances of the class.
- **__init__ method:** The constructor, called automatically when a new instance is created. It initializes the instance attributes.
- **self:** Refers to the current instance of the class. It's used to access attributes and methods of the class.
- **Instance Attributes:** Variables specific to each instance, initialized in the constructor.
- **Method:** A function defined within a class that can operate on the data (attributes) of that class.

Example 1:

```
class Rectangle:
    def __init__(self, length, width):
        self.length = length # Instance attribute: length of the rectangle
        self.width = width   # Instance attribute: width of the rectangle

    def area(self):
        return self.length * self.width # Calculates the area of the rectangle

    def perimeter(self):
        return 2 * (self.length + self.width) # Calculates the perimeter of the rectangle
```

Explanation:

- **Class Name:** Rectangle defines the blueprint for a rectangle.
- **Instance Attributes:** length and width are initialized in the constructor `__init__` and are unique to each instance.
- **Methods:**
 - **area():** Calculates and returns the area of the rectangle.
 - **perimeter():** Calculates and returns the perimeter of the rectangle.

Example 2:

```
class BankAccount:
    def __init__(self, owner, balance=0):
        self.owner = owner      # Instance attribute: name of the account owner
        self.balance = balance  # Instance attribute: initial balance (default is 0)

    def deposit(self, amount):
```

```
self.balance += amount    # Adds the deposit amount to the balance
return self.balance

def withdraw(self, amount):
    if amount > self.balance:
        return "Insufficient funds" # Checks if the withdrawal amount is greater than
the balance
    else:
        self.balance -= amount    # Subtracts the withdrawal amount from the
balance
    return self.balance
```

Explanation:

- **Class Name:** BankAccount defines a template for a bank account.
- **Instance Attributes:**
 - **owner:** Represents the owner of the account.
 - **balance:** Represents the balance in the account, with a default value of 0.
- **Methods:**
 - **deposit(amount):** Adds money to the account and updates the balance.
 - **withdraw(amount):** Deducts money from the account, if there are sufficient funds, and updates the balance.

III. Creating Objects

Creating objects from a class involves instantiating the class, which means creating an instance of it. Here's a simple guide with examples:

Syntax for Creating Objects:

- *object_name = ClassName("value1", "value2")*

Syntax for accessing the methods and attributes:

- *object_name.method_name()*
- *object_name.attributename*

Example 1: Basic Example

```
class Dog:
    def __init__(self, name, age):
        self.name = name
        self.age = age
```

```
def description(self):
    return f"{self.name} is {self.age} years old."

# Creating an instance of Dog
my_dog = Dog("Buddy", 5)

# Accessing attributes and methods
print(my_dog.name)      # Output: Buddy
print(my_dog.age)       # Output: 5
print(my_dog.description()) # Output: Buddy is 5 years old.
```

Explanation:

- Dog("Buddy", 5) creates an object of the Dog class with name set to "Buddy" and age set to 5.
- my_dog is the reference to this instance.

Output:

```
Buddy
5
Buddy is 5 years old.
```

Example 2: Class with Class Attribute

```
class Car:
    wheels = 4 # Class attribute

    def __init__(self, make, model):
        self.make = make
        self.model = model

    def car_info(self):
        return f"{self.make} {self.model} with {Car.wheels} wheels."

# Creating instances of Car
car1 = Car("Toyota", "Corolla")
car2 = Car("Honda", "Civic")

# Accessing attributes and methods
print(car1.car_info()) # Output: Toyota Corolla with 4 wheels.
print(car2.car_info()) # Output: Honda Civic with 4 wheels.
```

Explanation:

- Car("Toyota", "Corolla") creates an object of the Car class with make set to "Toyota" and model set to "Corolla".
- Car.wheels is a class attribute, shared by all instances of the Car class.

Output:

Toyota Corolla with 4 wheels.
Honda Civic with 4 wheels.

Example 3: Creating Multiple Objects

```
class Book:
    def __init__(self, title, author):
        self.title = title
        self.author = author

    def details(self):
        return f"{self.title}' by {self.author}"

# Creating multiple instances of Book
book1 = Book("1984", "George Orwell")
book2 = Book("To Kill a Mockingbird", "Harper Lee")

# Accessing attributes and methods
print(book1.details()) # Output: '1984' by George Orwell
print(book2.details()) # Output: 'To Kill a Mockingbird' by Harper Lee
```

Explanation:

- Each Book object is created with specific title and author values.
- book1 and book2 are separate instances of the Book class with their own attributes.

Output

'1984' by George Orwell
'To Kill a Mockingbird' by Harper Lee

VI. Class Attributes and Methods

Class attributes and methods are associated with the class itself rather than any particular instance of the class. They are shared among all instances of the class.

Class Attributes:

A class attribute is a variable that is declared within the class itself, outside of any instance methods. All instances of the class share the same class attribute.

Example: Class Attribute

```
class Circle:
    # Class attribute
    pi = 3.14159

    def __init__(self, radius):
        self.radius = radius # Instance attribute

    def area(self):
        # Accessing the class attribute
        return Circle.pi * self.radius ** 2

# Creating instances of Circle
circle1 = Circle(5)
circle2 = Circle(10)

# Accessing class attribute
print(circle1.pi) # Output: 3.14159
print(circle2.pi) # Output: 3.14159

# Accessing the area method
print(circle1.area()) # Output: 78.53975
print(circle2.area()) # Output: 314.159
```

Explanation:

- `pi` is a class attribute that is shared by all instances of the `Circle` class.
- Both `circle1` and `circle2` access the same value of `pi` when calculating the area.

Output:

```
3.14159
3.14159
78.53975
314.159
```

Class Methods

A class method is a method that is bound to the class rather than its objects. It can modify a class's state that applies across all instances of the class. Class methods are defined using the **@classmethod** decorator, and the first parameter is **cls**, which refers to the class itself.

Example: Class Method

```
class Employee:
    # Class attribute
    company_name = "TechCorp"

    def __init__(self, name, position):
        self.name = name      # Instance attribute
        self.position = position # Instance attribute

    @classmethod
    def change_company_name(cls, new_name):
        cls.company_name = new_name

    def display_employee_info(self):
        return f"{self.name} works as {self.position} at {Employee.company_name}"

# Creating instances of Employee
emp1 = Employee("Alice", "Developer")
emp2 = Employee("Bob", "Designer")

# Accessing class attribute
print(emp1.display_employee_info()) # Output: Alice works as Developer at TechCorp
print(emp2.display_employee_info()) # Output: Bob works as Designer at TechCorp

# Changing the class attribute using class method
Employee.change_company_name("InnoTech")

# Accessing class attribute after change
print(emp1.display_employee_info()) # Output: Alice works as Developer at InnoTech
print(emp2.display_employee_info()) # Output: Bob works as Designer at InnoTech
```

Explanation:

- **company_name** is a class attribute that is shared among all instances of the **Employee** class.

- The **change_company_name** method is a class method that updates the `company_name` attribute for all instances of the class.
- After calling `Employee.change_company_name("InnoTech")`, all instances of `Employee` reflect the updated company name.

Output

```
Alice works as Developer at TechCorp
Bob works as Designer at TechCorp
Alice works as Developer at InnoTech
Bob works as Designer at InnoTech
```

VII. Instance Attributes and Methods

Instance attributes and methods are specific to each object (instance) of a class. They define the individual state and behavior of each object.

Instance Attributes

Instance attributes are variables that hold data specific to an instance of a class. They are defined within the `__init__` method (or other methods) and are accessed using `self`.

Example: Instance Attributes

```
class Car:
    def __init__(self, make, model, year):
        # Instance attributes
        self.make = make
        self.model = model
        self.year = year

    def car_info(self):
        return f"{self.year} {self.make} {self.model}"

# Creating instances of Car
car1 = Car("Toyota", "Camry", 2020)
car2 = Car("Honda", "Accord", 2018)

# Accessing instance attributes
print(car1.make) # Output: Toyota
```

```
print(car2.model) # Output: Accord

# Accessing instance method
print(car1.car_info()) # Output: 2020 Toyota Camry
print(car2.car_info()) # Output: 2018 Honda Accord
```

Explanation:

- **make, model, and year** are instance attributes that store data specific to each Car object.
- **car1** and **car2** are different instances of the Car class, each with its own values for make, model, and year.

Output

```
Toyota
Accord
2020 Toyota Camry
2018 Honda Accord
```

Instance Methods

Instance methods are functions defined inside a class that operate on the instance attributes. They typically access or modify the instance's state and are called using the object of the class.

Example: Instance Methods

```
class BankAccount:
    def __init__(self, owner, balance=0):
        # Instance attributes
        self.owner = owner
        self.balance = balance

    # Instance method to deposit money
    def deposit(self, amount):
        self.balance += amount
        return f"${amount} deposited. New balance: ${self.balance}"

    # Instance method to withdraw money
```

```
def withdraw(self, amount):
    if amount > self.balance:
        return "Insufficient funds"
    else:
        self.balance -= amount
        return f"${amount} withdrawn. New balance: ${self.balance}"
```

```
# Creating an instance of BankAccount
account = BankAccount("John Doe", 1000)
```

```
# Accessing instance methods
print(account.deposit(500)) # Output: $500 deposited. New balance: $1500
print(account.withdraw(200)) # Output: $200 withdrawn. New balance: $1300
print(account.withdraw(2000)) # Output: Insufficient funds
```

Explanation:

- **deposit and withdraw** are instance methods that modify the balance attribute of the BankAccount instance.
- Each call to these methods affects the specific instance's balance.

Output

```
$500 deposited. New balance: $1500
$200 withdrawn. New balance: $1300
Insufficient funds
```

Key Differences Between Instance and Class Attributes/Methods

- **Instance Attributes/Methods:** Specific to each instance of the class. They represent the state and behavior unique to each object.
- **Class Attributes/Methods:** Shared among all instances of the class. They represent the state and behavior common across all objects of the class.

When to Use Class Attributes and Methods

- **Class Attributes:** Use when you want a variable to be shared among all instances of a class. Examples include constants or settings that apply across all instances.
- **Class Methods:** Use when you need to change or access class-level attributes, especially when the operation should affect all instances.

When to Use Instance Attributes and Methods

- **Instance Attributes:** Use when you need each object to store its own data. For example, each car has its own make, model, and year, so these should be instance attributes.
- **Instance Methods:** Use when you want methods that interact with or modify the state of individual objects. For example, each bank account can perform deposits and withdrawals on its own balance.

Exercise:

Use GPT to write a program:

1. You're tasked with creating a Python class named Student for a school management system. The class should have an `__init__` method to initialize the student's name and grade. Implement a method called `update_grade` that allows updating the student's grade, and another method called `student_summary` that returns a string in the format "Student Name: [name], Grade: [grade]". Create a Student object, update the grade, and then display the student's summary.

After the ILT, the student has to do a 30 min live lab. Please refer to the Day 6 Lab doc in the LMS.